LE GLASSMORPHISM

APPLICATION DE LA DEMARCHE SCIENTIFIQUE

Boivin Pierre

1. Résumé

Ce document présente une démarche scientifique appliquée à l'optimisation des performances Web face aux tendances de design modernes, en particulier le *Glassmorphism*. L'utilisation de la propriété CSS backdrop-filter pour générer des flous dynamiques provoque des ralentissements majeurs sur les terminaux dépourvus d'accélération graphique dédiée.

L'étude évalue plusieurs solutions techniques (optimisation matérielle, dégradation gracieuse, illusion d'optique) via un protocole de test automatisé mesurant la fréquence d'images (FPS).

Les résultats démontrent que l'adaptation dynamique au matériel client reste la seule stratégie viable, certaines méthodes palliatives statiques s'avérant plus coûteuses en ressources que le calcul initial.

Mots-clés : *Performances Web, Glassmorphism, CSS, GPU, FPS*

2. Table des matières

1. Résumé 1
2. Table des matières 1
3. Table des illustrations 2
4. Glossaire 2
5. Définition de la problématique 3
6. Analyse de l'existant 6
7. État de l'art et concepts théoriques 8
8. Propositions de solutions 10
9. Méthode de validation et protocole expérimental 12
10. Résultats et Analyse 14
11. Conclusion et Perspectives 19
12. Bibliographie & Références Techniques 21
13. Index 22

3. Table des illustrations

Figure 1 : Exemple bouton avec Glassmorphism 3
Figure 2 : Aperçu de l'Arrière-plan du prototype..... 12
Figure 3 : Aperçu pendant le test Standard 14
Figure 4 : Aperçu pendant le test Opti. CSS..... 15
Figure 5 : Aperçu pendant le test Adaptatif 15
Figure 6 : Aperçu pendant le test Illusion 16
Figure 7 : Résultats des tests sur ordinateur 16
Figure 8 : Résultats des tests sur mobile..... 17

4. Glossaire

- **Glassmorphism :** Tendance de design d'interface utilisateur (UI) reposant sur des effets de translucidité, des bordures claires et un flou d'arrière-plan pour imiter l'apparence du verre.
- **backdrop-filter: blur() :** Propriété CSS native permettant d'appliquer un algorithme de flou dynamique sur les éléments situés visuellement derrière un calque transparent.
- **FPS (Frames Per Second) :** Unité de mesure évaluant la fluidité d'un affichage. La norme cible pour une navigation Web fluide est de 60 FPS.
- **Jank :** Terme technique désignant une saccade visuelle ou une perte de fluidité survenant lorsque le matériel n'arrive pas à calculer les images assez rapidement.
- **CPU (Central Processing Unit) :** Processeur principal de l'appareil, optimisé pour les calculs séquentiels (comme l'exécution du JavaScript ou le calcul de la mise en page).
- **GPU (Graphics Processing Unit) :** Processeur graphique doté d'une architecture parallélisée, idéal pour le rendu visuel et le calcul matriciel des filtres de flou.
- **DOM (Document Object Model) :** Représentation structurelle en arbre des éléments HTML d'une page Web, lue et manipulée par le navigateur.
- **WebGPU :** API de nouvelle génération permettant un accès bas niveau au GPU, réduisant la latence des filtres visuels.
- **Pipeline de rendu :** Séquence d'étapes (DOM > CSSOM > Layout > Paint > Composite) par laquelle le navigateur transforme le code en image.
- **Shaders :** Programmes exécutés par le GPU pour effectuer les calculs de pixels complexes requis par le Glassmorphism.
- **Convolution matricielle :** Opération mathématique appliquant un noyau de flou sur une grille de pixels pour obtenir l'effet de translucidité.

5. Définition de la problématique

1. Contexte technologique et esthétique

Historiquement dominé par le "Flat Design", le Web s'oriente aujourd'hui vers des interfaces reprenant les codes des systèmes d'exploitation natifs (iOS, Windows 11). Cette tendance, baptisée *Glassmorphism*, repose sur des effets de profondeur, de translucidité et de flou d'arrière-plan.

Techniquement, cette esthétique est rendue possible sur les navigateurs par l'adoption de la propriété CSS `backdrop-filter`. Cependant, contrairement à une couleur unie, l'application d'un flou dynamique nécessite des calculs mathématiques complexes appliqués en temps réel à chaque modification de la page ou lors du défilement.

Figure 1 : Exemple bouton avec Glassmorphism

2. Origine et formulation de la problématique

Pour qu'une application Web soit perçue comme fluide, le navigateur doit afficher 60 images par seconde (FPS), laissant ainsi moins de 16,6 millisecondes au matériel pour calculer chaque image.

Les filtres de flou requièrent l'intervention intensive du processeur graphique (GPU). Sur des appareils aux ressources limitées, ce surcoût entraîne des chutes drastiques de la fréquence d'images (*jank*), provoquant une navigation saccadée et une surconsommation énergétique.

La problématique scientifique de cette étude se formule ainsi :

Comment intégrer des effets de rendu visuels complexes (Glassmorphism) sans dégrader la fluidité et l'autonomie des terminaux aux capacités matérielles hétérogènes (mobiles et ordinateurs bureautiques) ?

3. Objectifs de l'étude

Cette démarche vise trois objectifs quantifiables.

Il s'agit en premier lieu de mesurer avec précision l'impact de la propriété `backdrop-filter` sur la charge CPU/GPU et le temps de rendu par image.

Il conviendra ensuite de déterminer le seuil de tolérance matériel, c'est-à-dire le nombre d'éléments floutés simultanément avant l'effondrement de la fluidité sous les 30 FPS.

Enfin, l'objectif final est de proposer des solutions techniques viables (optimisation matérielle CSS, design adaptatif, illusion d'optique) pour garantir des performances optimales.

4. Périmètre

L'étude englobe l'environnement Web exécuté sur l'ensemble des terminaux dépourvus d'une accélération graphique dédiée performante.

Si cela concerne majoritairement les smartphones et tablettes, le périmètre s'étend également aux ordinateurs de bureau à usage bureautique reposant sur des puces graphiques intégrées.

Les ordinateurs équipés de cartes graphiques puissantes ainsi que les applications natives (bénéficiant d'optimisations constructeurs directes) sont exclus de cette étude, la dégradation des performances y étant négligeable.

5. Enjeux

Les enjeux soulevés par cette étude sont multiples.

Sur le plan de l'accessibilité, il est impératif d'éviter la création d'une fracture numérique matérielle où seuls les terminaux haut de gamme bénéficient d'une interface utilisable.

D'un point de vue éco-conception (Green IT), la réduction des cycles d'horloge inutiles du GPU permet de limiter la chauffe et de préserver l'autonomie des batteries.

Enfin, l'enjeu lié à l'expérience utilisateur (UX) est critique, puisqu'une chute de fluidité est directement corrélée à un taux d'abandon accru.

6. Contraintes et paramètres influents

La difficulté majeure de cette analyse réside dans l'hétérogénéité des moteurs de rendu des navigateurs et des capacités matérielles.

Par ailleurs, l'écosystème d'Apple réserve l'analyse des performances de Safari iOS aux seuls utilisateurs de Mac. Cette restriction matérielle a imposé une adaptation du protocole de test.

Afin de garantir la fiabilité des mesures, l'expérimentation combinera l'intégration d'outils de profilage de performance embarqués (via JavaScript) pour les relevés sur mobiles, et l'utilisation du débogage distant classique sur ordinateur pour valider l'impact matériel.

6. Analyse de l'existant

1. La situation actuelle du Web mobile

La consultation d'applications Web s'effectue aujourd'hui sur une multitude de terminaux, allant du smartphone à l'ordinateur.

Cependant, ce parc matériel est caractérisé par une fragmentation extrême.

Lors de la conception, les designers et développeurs travaillent généralement sur des machines de bureau très performantes et valident souvent leurs interfaces sur des smartphones haut de gamme. Ce processus crée un biais de perception technique.

Dans la réalité, l'utilisateur médian navigue sur un smartphone de milieu de gamme ou sur un ordinateur de bureau classique équipé d'un simple processeur graphique intégré.

L'intégration de tendances exigeantes comme le Glassmorphism, sans adaptation, impose un standard visuel unique à un parc matériel inégal.

Cette approche pénalise une large part des utilisateurs, créant une forme d'exclusion technologique basée sur le matériel.

2. Les causes du problème

La principale cause des baisses de performance réside dans la nature même du navigateur Web.

Contrairement à une application native qui communique presque directement avec les composants matériels du terminal, le navigateur agit comme une lourde couche d'abstraction.

Pour afficher une page, le moteur de rendu doit interpréter le code, construire la structure, calculer les styles, puis organiser la mise en page avant de dessiner les pixels. Lorsqu'un flou dynamique est appliqué via CSS, le navigateur ne peut plus dessiner l'élément une seule fois.

À chaque milliseconde d'animation, il est forcé de lire les pixels en arrière-plan, d'appliquer un algorithme de convolution mathématique, puis de fusionner le résultat.

Cette charge répétitive provoque un goulot d'étranglement entre le processeur principal et le processeur graphique.

3. Les contraintes matérielles et logicielles

Le problème de calcul est aggravé par les contraintes physiques des appareils.

Les smartphones, dépourvus de ventilation active, chauffent rapidement face à ces calculs graphiques en boucle. Le système déclenche alors un bridage thermique (ralentissement du processeur), détériorant brutalement la fluidité.

Sur les ordinateurs de bureau standards, bien que ventilés, c'est l'absence de carte graphique dédiée qui entraîne une saturation immédiate des ressources. Du côté logiciel, le paysage est tout aussi complexe. Sur les appareils Apple, le moteur de rendu bénéficie d'une intégration profonde et optimisée avec le matériel, masquant souvent le coût réel des effets visuels.

À l'inverse, dans l'écosystème Android et Windows, le moteur de rendu doit rester générique pour fonctionner sur des milliers de processeurs différents, l'empêchant de tirer parti d'optimisations spécifiques. Ainsi, un flou parfaitement fluide sur iOS peut totalement saturer un appareil concurrent de prix équivalent.

7. État de l'art et concepts théoriques

1. Le pipeline de rendu d'un navigateur Web

Pour comprendre l'impact d'un effet visuel sur les performances, il est impératif de décomposer le processus par lequel un navigateur transforme des lignes de code HTML et CSS en pixels à l'écran. Ce processus, appelé chemin critique de rendu, suit une séquence d'étapes très précises.

Le navigateur commence par analyser le balisage pour construire le modèle objet de document (DOM) et le modèle objet CSS (CSSOM). Ces deux structures sont ensuite combinées pour former l'arbre de rendu, qui ne contient que les éléments visibles sur la page.

Une fois cet arbre constitué, le moteur de rendu entame la phase de disposition, calculant la géométrie exacte de chaque élément, sa taille et sa position spatiale. Vient ensuite l'étape de peinture, où le navigateur dessine les pixels de chaque nœud sur différentes couches de pixels indépendantes, de la même manière qu'un logiciel de retouche photo utilise des calques.

Enfin, la dernière étape est la composition. Le navigateur assemble toutes ces couches peintes dans le bon ordre pour afficher l'image finale. C'est précisément lors de cette étape de composition que la propriété de filtre d'arrière-plan intervient, obligeant le moteur à fusionner visuellement la couche supérieure avec les pixels de la couche inférieure après leur avoir appliqué une transformation.

2. La délégation matérielle : CPU contre GPU

L'affichage fluide d'une page Web repose sur une répartition stricte des tâches entre le processeur principal (CPU) et le processeur graphique (GPU).

Le CPU est optimisé pour les calculs séquentiels complexes. Dans le contexte du Web, il se charge de l'exécution du JavaScript, du calcul de la mise en page et de la préparation des calques lors de l'étape de peinture.

Le GPU, quant à lui, possède une architecture hautement parallélisée, idéale pour manipuler des matrices de pixels simultanément.

Afin de garantir une animation fluide à soixante images par seconde, chaque image doit être calculée en moins de 16.67 millisecondes.

Pour atteindre cette cible, les navigateurs modernes tentent de déléguer un maximum d'opérations visuelles au GPU grâce à l'accélération matérielle. Lors de la phase de

composition, le CPU téléverse les calques sous forme de textures dans la mémoire vidéo (VRAM). Le GPU applique ensuite les transformations, les opacités et les filtres. Si l'application d'un flou est déléguée au GPU, ce dernier utilise des programmes spécifiques appelés "shaders" pour calculer le nouveau rendu.

Sur des appareils mobiles d'entrée de gamme, la bande passante entre le CPU et le GPU, ainsi que la puissance de calcul brute des unités d'exécution graphiques, constituent le principal goulot d'étranglement face à ce type de charge.

3. Modélisation mathématique et complexité du Flou Gaussien

Le rendu esthétique du Glassmorphism repose majoritairement sur l'algorithme du flou gaussien.

Contrairement à une simple modification de couleur, cet algorithme calcule la valeur de chaque pixel en effectuant une moyenne pondérée des pixels environnants. Le poids de chaque pixel voisin est déterminé par une fonction mathématique bidimensionnelle basée sur la distribution normale, dont l'équation s'exprime ainsi :

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Dans cette équation, x et y représentent la distance par rapport au pixel central, et la variable σ (l'écart-type) détermine l'intensité du flou, correspondant directement à la valeur en pixels définie dans la propriété CSS `backdrop-filter: blur()`.

D'un point de vue informatique, l'application de cette formule exige de superposer une matrice de convolution sur l'image source.

Plus le rayon du flou est grand, plus la matrice s'élargit. Pour une image de largeur W et de hauteur H, avec un rayon de flou R, la complexité algorithmique naïve s'élève à $O(W \times H \times R^2)$.

Bien que les moteurs de rendu modernes optimisent cette opération en séparant le flou bidimensionnel en deux passes unidimensionnelles successives, réduisant la complexité théorique à $O(W \times H \times R)$, le coût en opérations flottantes reste colossal.

Lorsqu'un utilisateur fait défiler la page, l'arrière-plan changeant force le GPU à recalculer l'intégralité de cette matrice à chaque nouvelle image générée, ce qui explique l'effondrement immédiat des performances sur les puces graphiques limitées.

8. Propositions de solutions

Au regard des contraintes physiques et algorithmiques identifiées précédemment, l'abandon pur et simple du Glassmorphism constituerait une réponse de facilité face à un défi d'ingénierie.

L'objectif de cette étude n'est donc pas de proscrire ces interfaces modernes, mais d'établir des stratégies d'implémentation techniquement viables.

Pour résoudre ce dilemme entre exigence esthétique et performance matérielle, trois hypothèses de résolution complémentaires ont été formulées. Elles explorent respectivement l'optimisation "bas niveau" du moteur de rendu, l'adaptation dynamique au matériel client, et la substitution par illusion d'optique.

1. Piste 1 : L'optimisation CSS

La première approche consiste à soulager le processeur principal en forçant le navigateur à isoler l'élément utilisant le flou sur son propre calque de rendu matériel.

Par défaut, le moteur de composition peut décider de regrouper plusieurs éléments du DOM sur un même calque graphique pour économiser de la mémoire.

Cependant, si un de ces éléments subit une modification visuelle ou spatiale, l'intégralité du calque doit être repeinte, ce qui multiplie les calculs de convolution du flou.

Pour forcer la ségrégation matérielle, la solution proposée repose sur l'utilisation de la propriété CSS `will-change: transform` ou sur l'application d'une transformation tridimensionnelle nulle, telle que `transform: translateZ(0)`.

Cette technique indique explicitement au navigateur que l'élément doit être traité indépendamment par le processeur graphique (GPU). Ainsi, lors d'un défilement de page, le moteur de rendu se contente de déplacer les textures déjà calculées dans la mémoire vidéo, limitant drastiquement les opérations de repeinte et réduisant la chute de la fréquence d'images.

2. Piste 2 : Conception adaptative conditionnelle

Puisqu'il est impossible de garantir une exécution fluide du flou gaussien sur des appareils anciens, la deuxième solution s'appuie sur le principe de dégradation gracieuse.

L'objectif est de concevoir l'interface pour détecter les limites matérielles ou les préférences du système d'exploitation de l'utilisateur, afin de désactiver dynamiquement

l'effet de transparence lourde au profit d'une couleur opaque ou semi-transparente basique.

Cette détection s'opère à deux niveaux :

- Le premier niveau utilise les requêtes multimédias CSS (Media Queries), spécifiquement l'instruction `@media (prefers-reduced-transparency)`. Cette directive permet de cibler les utilisateurs ayant volontairement activé les options d'accessibilité ou d'économie d'énergie de leur système d'exploitation.

- Le second niveau nécessite l'intervention de JavaScript pour évaluer directement la puissance matérielle du terminal client. En interrogeant l'API du navigateur, il est possible de récupérer le nombre de cœurs logiques du processeur via `navigator.hardwareConcurrency` ou la quantité de mémoire vive via `navigator.deviceMemory`. Si les valeurs retournées sont inférieures à un seuil critique prédéfini, l'application Web retire automatiquement la classe CSS responsable du Glassmorphism, garantissant ainsi le maintien d'une fréquence de soixante images par seconde.

3. Piste 3 : L'illusion d'optique

La troisième hypothèse vise à éliminer totalement le calcul mathématique du flou en temps réel tout en conservant l'esthétique du Glassmorphism.

Cette technique repose sur une illusion d'optique utilisant des images précalculées. Plutôt que de demander au navigateur d'appliquer un filtre sur les éléments situés derrière une carte, la solution consiste à générer en amont une version de l'image d'arrière-plan déjà floutée (par exemple via un format optimisé comme le WebP) et à l'appliquer comme image de fond statique sur l'élément superposé.

En ajustant précisément le positionnement de cette image d'arrière-plan via la propriété `background-attachment: fixed` et en synchronisant les coordonnées, l'élément donne l'illusion de déformer ce qui se trouve derrière lui.

Cette méthode annule la complexité algorithmique du filtre dynamique et supprime le goulot d'étranglement de l'étape de composition matérielle.

Bien que cette solution soit contraignante techniquement lors de la phase de développement et qu'elle augmente légèrement le poids de téléchargement de la page initiale, elle garantit des performances maximales et constantes, quelle que soit la vétusté du processeur graphique de l'appareil client.

9. Méthode de validation et protocole expérimental

1. L'environnement de test (Le prototype)

Pour pouvoir comparer nos différentes solutions de manière fiable, il fallait éviter de faire nos tests sur un site web complet où d'autres éléments (images lourdes, scripts tiers) pourraient fausser les résultats.

Nous avons donc développé une petite application web sur mesure.

Le principe de ce prototype est simple :

- Un arrière-plan complexe et animé en permanence (des halos lumineux qui se déplacent) a été mis en place pour s'assurer que le processeur graphique soit sollicité.
- Par-dessus cet arrière-plan, une interface permet de générer dynamiquement des dizaines de "cartes" et de basculer instantanément entre les 4 solutions techniques que nous voulons tester (Standard, Optimisation CSS, Conception adaptative conditionnelle, et Illusion d'optique).

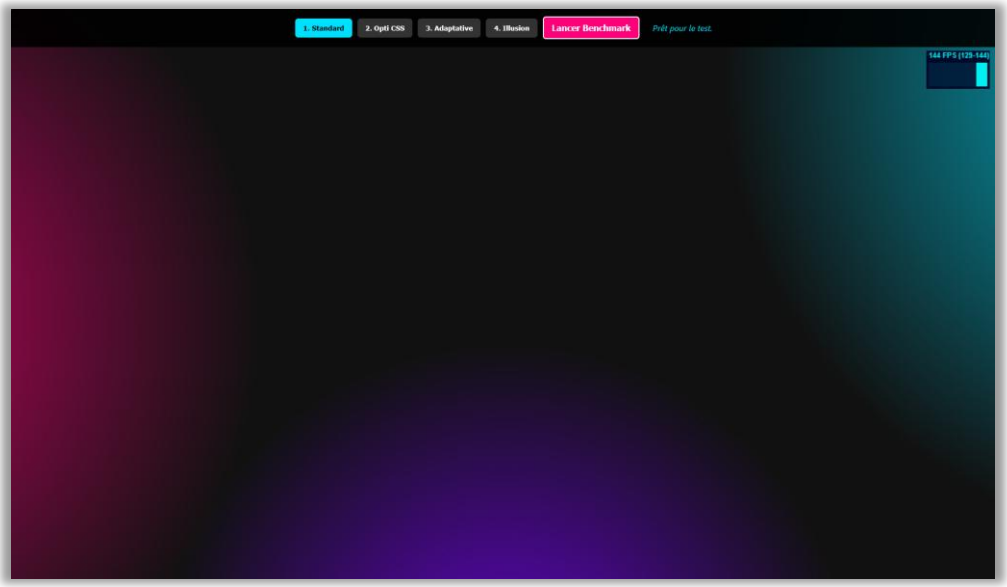

Figure 2 : Aperçu de l'Arrière-plan du prototype

Ce banc de test minimaliste nous permet d'être certains que les baisses de performances que nous allons observer seront uniquement dues aux effets de transparence et de flou.

2. L'indicateur de performance choisi (Le KPI)

Afin de refléter fidèlement l'expérience utilisateur, nous avons choisi de nous concentrer sur une métrique principale : le Frame Rate, ou nombre d'images par seconde (FPS).

Sur le Web, une animation ou un défilement (scroll) est considéré comme parfaitement fluide s'il maintient la cadence de 60 FPS. Dès que ce chiffre tombe en dessous des 30 FPS, l'œil humain perçoit des saccades et l'interface donne une impression de lenteur.

Notre objectif lors des tests sera donc d'observer à partir de combien de cartes affichées l'appareil n'arrive plus à tenir ces fameux 60 FPS.

3. L'automatisation des mesures

Au lieu de rajouter des cartes manuellement et d'essayer de lire un compteur à l'œil nu (ce qui manquerait de précision), nous avons fait le choix de développer un script de *Benchmark* (banc d'essai) entièrement automatisé en JavaScript.

Une fois lancé, ce script prend le contrôle de la page :

Il applique la première solution technique (Flou standard).

Il affiche 10 cartes à l'écran, attend une demi-seconde que le navigateur se stabilise, puis compte précisément le nombre d'images affichées pendant la seconde qui suit grâce à la fonction native `requestAnimationFrame`.

Il enregistre le score, passe à 20 cartes, puis 30, jusqu'à 80.

Il nettoie l'écran et recommence le même processus pour la solution 2, puis la 3, etc.

À la fin du test, les données récoltées sont instantanément transformées en un graphique visuel grâce à la bibliothèque *Chart.js*. Cette méthode garantit que chaque solution est testée exactement dans les mêmes conditions, sans intervention humaine susceptible de biaiser le test.

4. Matériel testé et conditions

Pour avoir des résultats représentatifs, les tests n'ont pas été réalisés sur des émulateurs, mais sur des appareils physiques réels. Le prototype a notamment été exécuté sur un *ASUS ROG* avec *Firefox* pour représenter une utilisation bureautique standard, et sur un IPhone 13 pour représenter l'usage sur smartphone.

Afin de garantir la fiabilité des relevés, chaque test a été lancé dans des conditions identiques : appareil branché sur secteur (pour éviter les modes d'économie d'énergie automatiques de la batterie), luminosité fixe, et sans aucun autre onglet ou application gourmande fonctionnant en arrière-plan.

10. Résultats et Analyse

1. Analyse visuelle des différentes implémentations

Avant d'étudier les chiffres, il est essentiel d'observer l'impact de nos solutions sur le rendu visuel. Les captures d'écran suivantes permettent de comparer la fidélité de l'interface selon la méthode choisie.

L'image ci-dessous présente l'interface de référence lors du test du Flou Standard. L'effet de Glassmorphism est calculé en temps réel par-dessus l'arrière-plan animé.

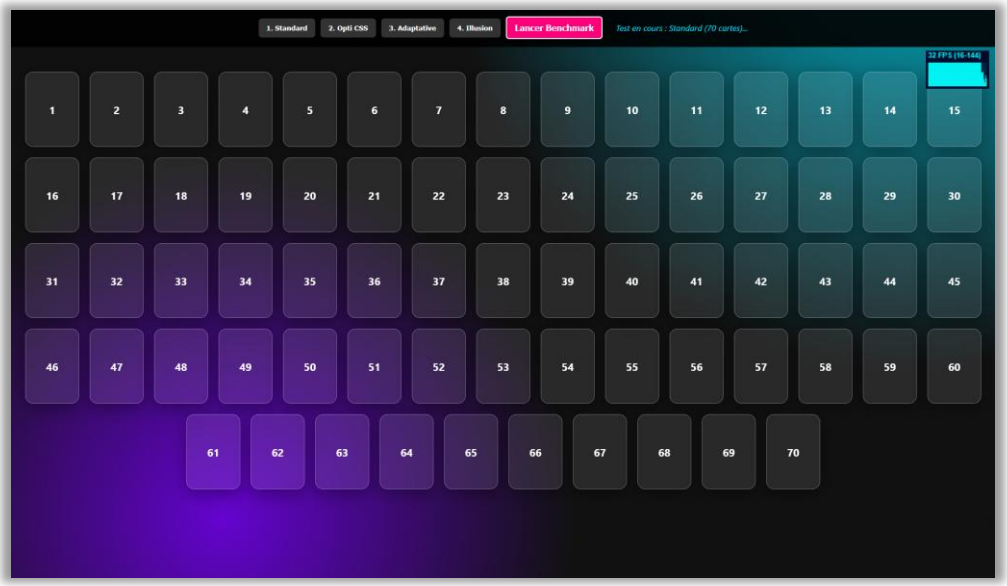

Figure 3 : Aperçu pendant le test Standard

L'application de l'optimisation matérielle via CSS ne modifie en rien l'esthétique de l'interface comme on peut le constater ci-dessous. Le rendu visuel reste strictement identique à l'affichage standard, validant ainsi l'aspect transparent de cette solution pour l'utilisateur, qui ne perçoit aucune différence.

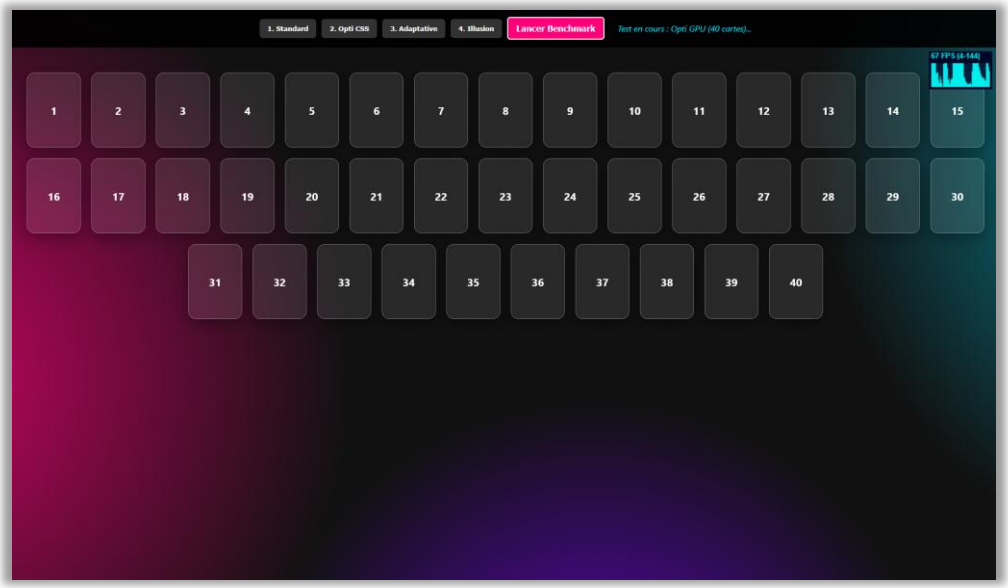

Figure 4 : Aperçu pendant le test Opti. CSS

L'image ci-dessous illustre le déclenchement de la solution de Conception adaptative conditionnelle. Lorsque l'appareil manque de ressources, l'effet de flou est désactivé au profit d'un fond opaque (ici configuré en blanc par défaut).

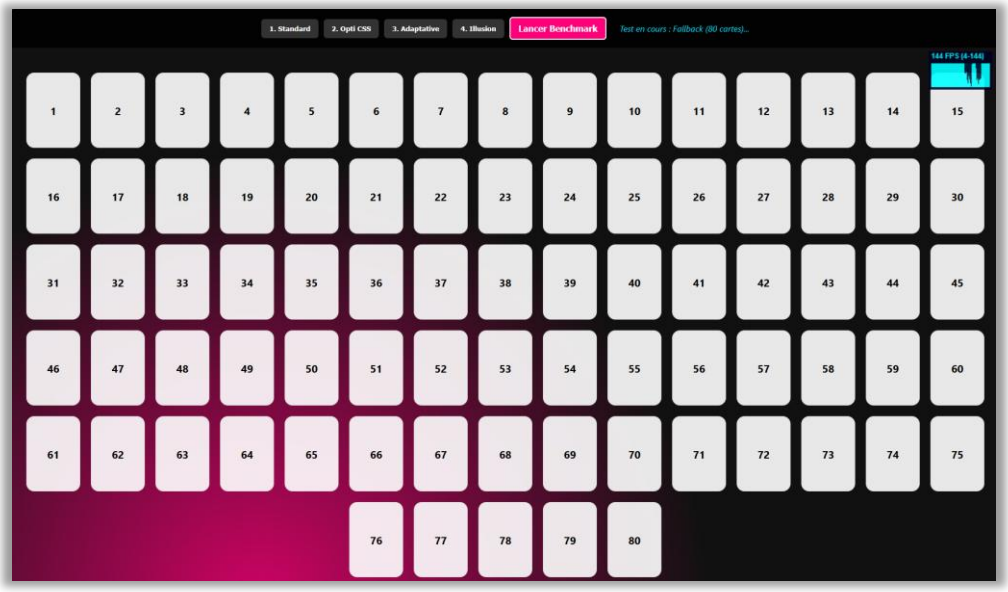

Figure 5 : Aperçu pendant le test Adaptatif

Bien que la rupture esthétique avec l'arrière-plan soit très marquée dans ce prototype, ce comportement est modulaire : en production, l'utilisation d'une couleur sombre ou semi-transparente permettrait d'atténuer ce contraste.

Enfin, le test de l'Illusion d'optique parvient à maintenir l'esthétique visuelle du Glassmorphism. Toutefois, l'image pré-floutée utilisée pour texturer les cartes étant par

nature statique, elle se désynchronise de l'arrière-plan animé, ce qui trahit visuellement l'illusion.

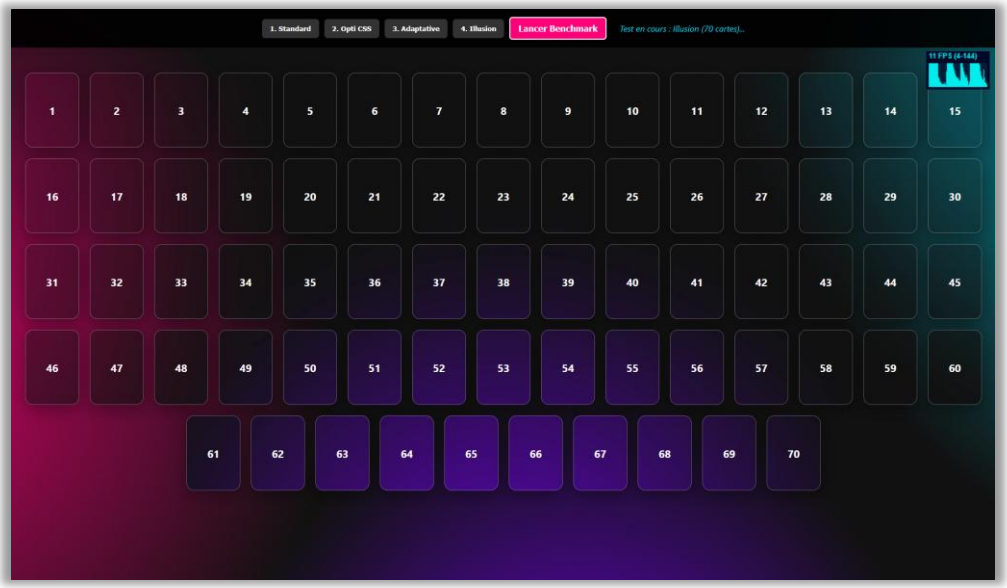

Figure 6 : Aperçu pendant le test Illusion

2. Interprétation des données : PC vs Mobile

– **Sur Ordinateur (PC) :**

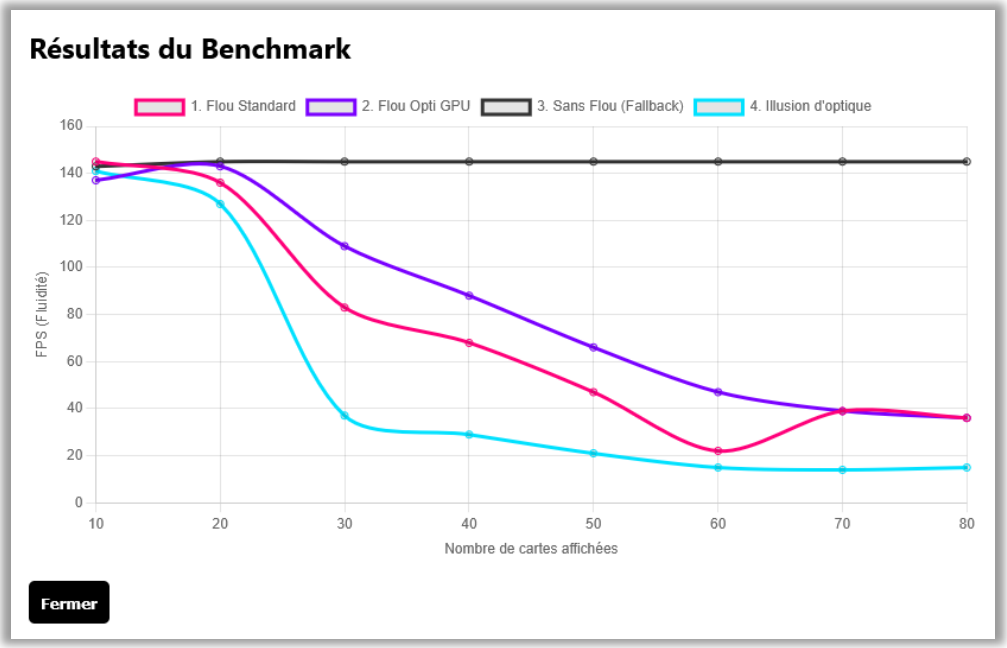

Figure 7 : Résultats des tests sur ordinateur

Sur PC, le problème principal constaté est la saturation du moteur de rendu dès que le nombre d'éléments augmente. Avec le « Flou Standard », on observe une chute progressive de la fluidité à mesure que l'on ajoute des cartes.

Cela prouve que même sur un ordinateur, le calcul mathématique du flou sur une grande surface reste une opération lourde pour les puces graphiques intégrées. L'optimisation GPU permet de stabiliser les résultats, mais c'est la solution de « Conception adaptative conditionnelle » qui garantit un maintien constant à 60 FPS.

Cependant ce n'est pas la solution qui reproduit au mieux le visuel de référence.

– **Sur Smartphone (iPhone) :**

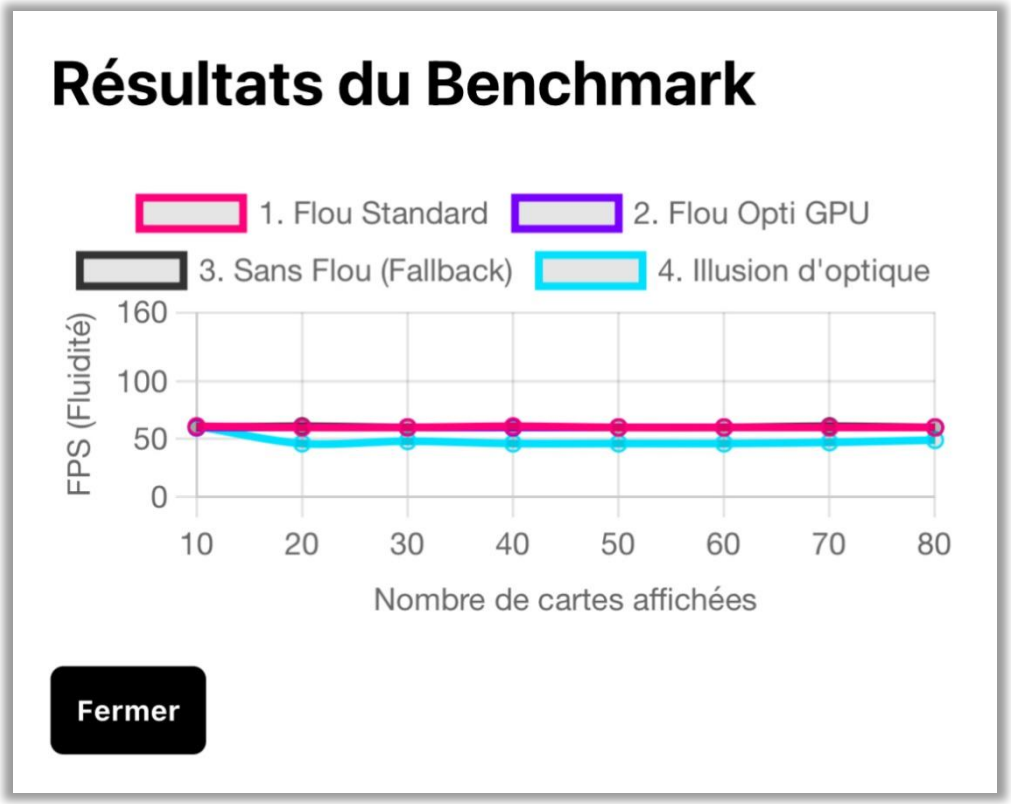

Figure 8 : Résultats des tests sur mobile

Les résultats sur iPhone sont plus surprenants. La puissance brute de la puce permet de maintenir le Flou Standard à 60 FPS sans difficulté majeure. En revanche, c'est l'Illusion d'optique qui affiche les moins bonnes performances, avec une chute aux alentours de 40 FPS.

Ce résultat, bien que contre-intuitif (une image étant censée être plus légère qu'un calcul de flou), s'explique techniquement par l'utilisation de la propriété `background-attachment: fixed`.

Pour maintenir l'image fixe pendant le mouvement, le navigateur doit recalculer l'affichage de chaque pixel à chaque frame, ce qui sature le processeur de l'iPhone.

3. Synthèse : Le compromis entre design et performance

L'analyse de ces résultats nous permet de tirer des conclusions concrètes pour le développement d'interfaces modernes :

L'illusion n'est pas une solution miracle : Ce qui semble être une astuce légère peut s'avérer plus lourd pour le navigateur que le calcul réel du flou, surtout sur mobile.

L'importance de la détection : La solution la plus fiable reste le mode « Adaptatif ». Un bon site doit être capable de détecter si l'appareil "rame" pour désactiver automatiquement le flou et passer sur une couleur unie afin d'éviter l'exclusion numérique matérielle.

L'optimisation invisible : L'ajout de propriétés comme `will-change` doit devenir un réflexe de développement, car elle permet d'améliorer la fluidité sans jamais altérer le design voulu par les graphistes.

11. Conclusion et Perspectives

1. Bilan de l'étude

La problématique de cette étude visait à déterminer comment intégrer des effets visuels lourds, tels que le Glassmorphism, sans dégrader les performances des terminaux mobiles et bureautiques aux capacités hétérogènes.

L'expérimentation a permis de mettre en lumière des comportements techniques contre-intuitifs, notamment le fait que les solutions basées sur des illusions d'optique statiques s'avèrent plus coûteuses en ressources CPU que le calcul mathématique du flou lui-même sur les navigateurs mobiles.

Il ressort de cette analyse que la contrainte matérielle ne doit pas être perçue comme un frein à la créativité, mais comme un paramètre d'ingénierie à part entière. Le maintien d'une fréquence de soixante images par seconde n'est pas incompatible avec les tendances design modernes, à condition de renoncer à l'idée d'un rendu universellement identique.

L'approche la plus viable industriellement reste l'adaptation conditionnelle : utiliser l'accélération matérielle (`will-change`) pour soulager les appareils intermédiaires, et implémenter des mécanismes d'adaptation (suppression du flou) pour les terminaux détectés ayant une puissance insuffisantes.

2. Recommandations pour la conception Web

À l'issue de ces travaux, plusieurs bonnes pratiques peuvent être établies pour les équipes de production :

Briser le silo Design/Développement : Les choix esthétiques (comme le contraste de l'arrière-plan ou l'intensité du flou) ont un impact direct sur le pipeline de rendu. Une collaboration étroite dès la phase de maquettage est indispensable pour éviter de concevoir des interfaces techniquement irréalisables à grande échelle.

Adopter l'amélioration progressive : Le développement doit prioriser une interface fluide, opaque et accessible. Le Glassmorphism ne doit être injecté qu'en seconde intention, si le navigateur confirme sa capacité à le traiter sans latence.

Diversifier les environnements de test : L'évaluation des performances Web ne doit plus s'effectuer exclusivement sur des ordinateurs de bureau ou des smartphones haut de gamme, sous peine de valider des produits inaccessibles pour la majorité des utilisateurs.

3. Perspectives technologiques

Le Web est un environnement en mutation constante. Si le calcul du flou dynamique pose aujourd'hui un défi d'optimisation, les évolutions à court terme du standard Web pourraient redéfinir cette problématique.

L'arrivée progressive de l'API WebGPU dans les navigateurs modernes promet un accès beaucoup plus direct et bas niveau à la carte graphique, réduisant considérablement la charge d'abstraction du navigateur.

Par ailleurs, l'intégration croissante de l'intelligence artificielle et des puces dédiées (NPU) dans les smartphones d'entrée de gamme ouvre de nouvelles perspectives. Il est envisageable que les futurs moteurs de rendu délèguent le traitement des filtres visuels complexes à ces unités neuronales, rendant ainsi le coût du Glassmorphism transparent pour l'architecture matérielle globale.

D'ici là, l'adaptation dynamique et le compromis technique restent les meilleurs outils de l'ingénieur.

12. Bibliographie & Références Techniques

Cette bibliographie regroupe les ressources fondamentales ayant permis d'étayer les analyses mathématiques et les observations techniques de ce rapport.

Fondements du Rendu Web et Pipeline :

- **Google Developers – Web Fundamentals.** *Critical Rendering Path*. Analyse détaillée des étapes du moteur de rendu : de la construction du DOM/CSSOM à la composition finale (Compositing).
    - Lien : https://web.dev/articles/critical-rendering-path
- **Intel Software.** *Web Graphics Performance*. Étude sur la délégation des calculs au processeur graphique (GPU) et l'impact de l'accélération matérielle sur le "repaint" du navigateur.
    - Lien : https://www.intel.com/content/www/us/en/developer/articles/guide/web-graphics-performance.html

Mathématiques et Algorithmique du Flou :

- **W3C Working Group.** *Filter Effects Module Level 1*. Spécifications techniques du flou gaussien et son implémentation via la distribution normale.
    - Lien : https://www.w3.org/TR/filter-effects-1/#feGaussianBlurElement
- **Mozilla (MDN Web Docs).** *Performance et CSS : Backdrop-filter*. Guide sur le coût de calcul des filtres de flou dynamique sur les calques de composition.
    - Lien : https://developer.mozilla.org/fr/docs/Web/CSS/backdrop-filter

Outils et Méthodologie :

- **Chrome DevTools Documentation.** *Evaluate performance*. Protocole d'utilisation du "Performance Monitor" pour le relevé des FPS et la détection du bridage thermique.
    - Lien : https://developer.chrome.com/docs/devtools/performance

- Ce rapport a été rédigé avec l'assistance de l'intelligence artificielle **Gemini**. L'outil a été utilisé comme une aide à la rédaction pour structurer le plan, corriger la syntaxe et m'aider à formuler clairement mes analyses techniques, tout en m'assurant d'utiliser le bon vocabulaire professionnel.
  La réflexion, les expérimentations et les résultats présentés restent, bien entendu, le fruit de mon travail personnel.
    - Lien : https://gemini.google.com/

13. Index

Code du protocole de test, aussi disponible via ce lien :
https://pierrebvn.github.io/Glossmorphism/

```
<!DOCTYPE html>

<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Laboratoire ADS - Benchmark Complet & Live FPS</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/stats.js/16/Stats.min.js"></script
>
    <style>
        /* Le fond complexe et ANIMÉ */
        body {
            margin: 0; min-height: 100vh;
            background-color: #111;
            background-image:
                radial-gradient(circle at 15% 50%, rgba(255, 0, 122, 0.8),
transparent 25%),
                radial-gradient(circle at 85% 30%, rgba(0, 225, 255, 0.8),
transparent 25%),
                radial-gradient(circle at 50% 80%, rgba(122, 0, 255, 0.8),
transparent 25%);
            background-size: 200% 200%; /* Nécessaire pour l'animation */
            animation: moveBlobs 10s ease-in-out infinite alternate;
            font-family: system-ui, sans-serif; padding-top: 100px;
        }

        /* Animation de l'arrière-plan */
        @keyframes moveBlobs {
            0% { background-position: 0% 0%; }
            100% { background-position: 100% 100%; }
        }

        #controls {
            position: fixed; top: 0; left: 0; right: 0;
            background: rgba(0,0,0,0.9); color: white;
            padding: 15px; z-index: 1000; display: flex; gap: 10px; flex-wrap:
wrap;
            align-items: center; justify-content: center;
        }
```

```
        button { padding: 10px 15px; border: none; border-radius: 6px;
background: #333; color: white; cursor: pointer; font-weight: bold;}
        button.active { background: #00e1ff; color: black; }
        .btn-run { background: #ff007a; font-size: 16px; border: 2px solid
white;}

        #card-container { display: flex; flex-wrap: wrap; gap: 20px; padding:
20px; justify-content: center; }

        .card { width: 100px; height: 140px; border-radius: 15px; display:
flex; align-items: center; justify-content: center; color: white; font-weight:
bold; font-size: 20px; }

        /* Les 4 états testés */
        .glass-standard { background: rgba(255, 255, 255, 0.1); backdrop-
filter: blur(20px); -webkit-backdrop-filter: blur(20px); border: 1px solid
rgba(255,255,255,0.3); box-shadow: 0 8px 32px 0 rgba(0,0,0,0.37); }
        .glass-optimized { background: rgba(255, 255, 255, 0.1); backdrop-
filter: blur(20px); -webkit-backdrop-filter: blur(20px); will-change:
transform; transform: translateZ(0); border: 1px solid rgba(255,255,255,0.3);
box-shadow: 0 8px 32px 0 rgba(0,0,0,0.37);}
        .glass-fallback { background: rgba(255, 255, 255, 0.9); color: black;
backdrop-filter: none; -webkit-backdrop-filter: none; border: 1px solid
rgba(0,0,0,0.1); }
        .glass-illusion { background-image: url('fond-flou2.webp');
background-size: cover; background-position: center; background-attachment:
fixed; border: 1px solid rgba(255, 255, 255, 0.3); box-shadow: 0 8px 32px 0
rgba(0, 0, 0, 0.37); }

        #chart-modal {
            display: none; position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%);
            width: 90%; max-width: 800px; background: white; padding: 20px;
border-radius: 15px;
            box-shadow: 0 10px 50px rgba(0,0,0,0.5); z-index: 2000;
        }
        #status-text { margin-left: 15px; font-style: italic; color: #00e1ff;}
    </style>
</head>
<body>

    <div id="controls">
        <button id="btn-standard" class="active" onclick="setMode('glass-
standard', this)">1. Standard</button>
        <button id="btn-optimized" onclick="setMode('glass-optimized',
this)">2. Opti CSS</button>
        <button id="btn-fallback" onclick="setMode('glass-fallback', this)">3.
Adaptative</button>
```

```
        <button id="btn-illusion" onclick="setMode('glass-illusion', this)">4.
Illusion</button>
        <button class="btn-run" onclick="runAutomatedBenchmark()">Lancer
Benchmark</button>
        <span id="status-text">Prêt pour le test.</span>
    </div>

    <div id="card-container"></div>

    <div id="chart-modal">
        <h2 style="color: black; margin-top: 0;">Résultats du Benchmark</h2>
        <canvas id="resultsChart"></canvas>
        <button style="margin-top: 15px; background: black; color: white;
padding: 10px; border: none; border-radius: 5px; cursor: pointer;"
onclick="document.getElementById('chart-
modal').style.display='none'">Fermer</button>
    </div>

    <script>
        // --- INITIALISATION DU COMPTEUR FPS EN TEMPS RÉEL (Stats.js) ---
        var stats = new Stats();
        stats.showPanel(0); // Affiche uniquement les FPS (0)
        document.body.appendChild(stats.dom);
        // en haut à droite pour ne pas cacher les boutons
        stats.dom.style.cssText =
'position:fixed;top:80px;right:15px;cursor:pointer;opacity:0.9;z-
index:10000;transform:scale(1.5);transform-origin:top right;';

        // Boucle d'animation pour mettre à jour le compteur visuel en continu
        function animateStats() {
            stats.begin();
            stats.end();
            requestAnimationFrame(animateStats);
        }
        requestAnimationFrame(animateStats);
        // ----------------------------------------------------------------


        const container = document.getElementById('card-container');
        const statusText = document.getElementById('status-text');
        let currentClass = 'glass-standard';

        function setMode(className, buttonElement) {
            currentClass = className;
            document.querySelectorAll('#controls button:not(.btn-
run)').forEach(btn => btn.classList.remove('active'));
            if(buttonElement) buttonElement.classList.add('active');
```

```
            document.querySelectorAll('.card').forEach(card => card.className
= `card ${currentClass}`);
        }

        function setCards(amount) {
            container.innerHTML = '';
            for(let i = 0; i < amount; i++) {
                const card = document.createElement('div');
                card.className = `card ${currentClass}`;
                card.innerText = i + 1;
                container.appendChild(card);
            }
        }

        // Fonction de mesure mathématique pour le graphique
        function measureFPS() {
            return new Promise(resolve => {
                let frames = 0;
                let startTime = performance.now();
                function loop() {
                    frames++;
                    if (performance.now() - startTime < 1000) {
requestAnimationFrame(loop); }
                    else { resolve(frames); }
                }
                requestAnimationFrame(loop);
            });
        }

        async function runAutomatedBenchmark() {
            const steps = [10, 20, 30, 40, 50, 60, 70, 80];
            const results = { standard: [], optimized: [], fallback: [],
illusion: [] };

            const modes = [
                { id: 'glass-standard', name: 'Standard', array:
results.standard },
                { id: 'glass-optimized', name: 'Opti GPU', array:
results.optimized },
                { id: 'glass-fallback', name: 'Fallback', array:
results.fallback },
                { id: 'glass-illusion', name: 'Illusion', array:
results.illusion }
            ];

            for (let mode of modes) {
                setMode(mode.id);
                for (let amount of steps) {
```

```
                    statusText.innerText = `Test en cours : ${mode.name}
(${amount} cartes)...`;
                    setCards(amount);
                    await new Promise(r => setTimeout(r, 500));
                    const fps = await measureFPS();
                    mode.array.push(fps);
                }
            }

            statusText.innerText = "Benchmark terminé !";
            setCards(0);
            drawChart(steps, results);
        }

        let myChart = null;
        function drawChart(labels, data) {
            document.getElementById('chart-modal').style.display = 'block';
            const ctx =
document.getElementById('resultsChart').getContext('2d');
            if(myChart) myChart.destroy();

            myChart = new Chart(ctx, {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [
                        { label: '1. Flou Standard', data: data.standard,
borderColor: '#ff007a', tension: 0.3, fill: false },
                        { label: '2. Flou Opti GPU', data: data.optimized,
borderColor: '#7a00ff', tension: 0.3, fill: false },
                        { label: '3. Sans Flou (Fallback)', data:
data.fallback, borderColor: '#333333', tension: 0.3, fill: false },
                        { label: '4. Illusion d\'optique', data:
data.illusion, borderColor: '#00e1ff', tension: 0.3, fill: false }
                    ]
                },
                options: {
                    responsive: true,
                    scales: {
                        y: { beginAtZero: true, max: 160, title: { display:
true, text: 'FPS (Fluidité)' } },
                        x: { title: { display: true, text: 'Nombre de cartes
affichées' } }
                    }
                }
            });
        }
    </script>
```

```
</body>
</html>
```